CSC 4553

Fall 2026 - Day 19

Admin

Page replacement

Questions to consider

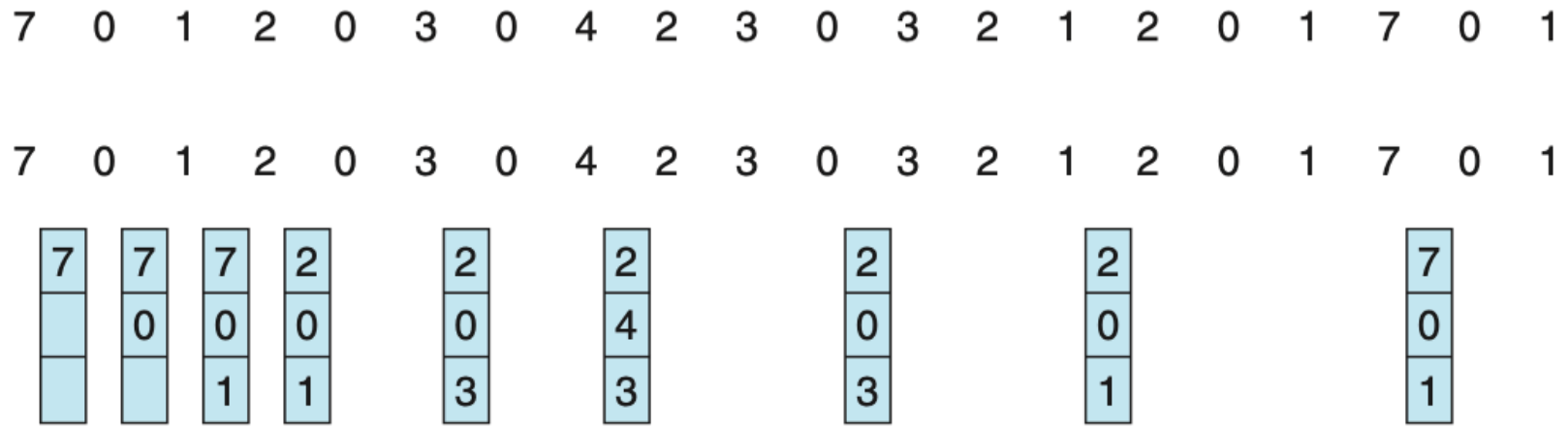
- What makes a good page replacement strategy, and why?
- How can we approximate optimal page replacement without knowing the future?
- What are the practical implementation challenges for different page replacement algorithms?

Page replacement

- Memory is full and a process needs a page: evict a frame (page replacement)
- The evicted page is the **victim**
- Costly: 1 I/O for a clean victim, 2 for a dirty one
- Clean pages are preferred victims
  - → Dirty pages need a write-back first
- Downside of evicting dirty pages?
  - → Forces the write at the worst time
  - → Dirty pages were recently active

Page replacement algorithms: OPT

- Optimal (OPT): evict the page that *won't be used for the longest time*
- Impossible (like SJF), but a useful benchmark
- Example: 3 frames, this reference pattern:



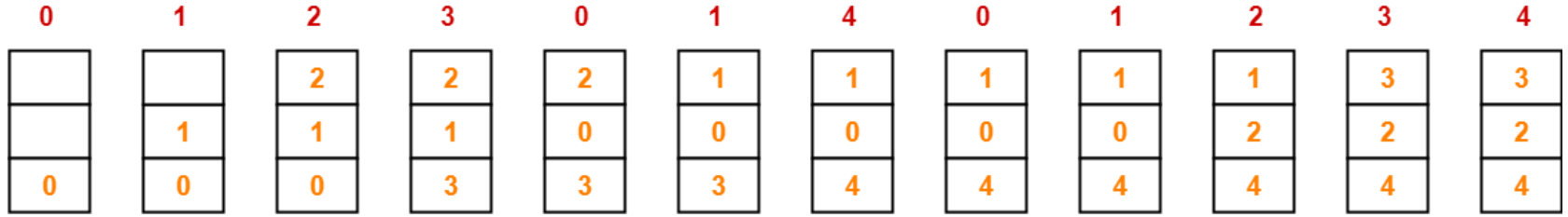
Page replacement algorithms: FIFO

- First-In-First-Out (FIFO): evict the oldest page
- Downsides?
  - → Ignores use: may evict important pages
- **Belady's anomaly**: more frames can mean more page faults
- Try this pattern with 3 frames, then 4:

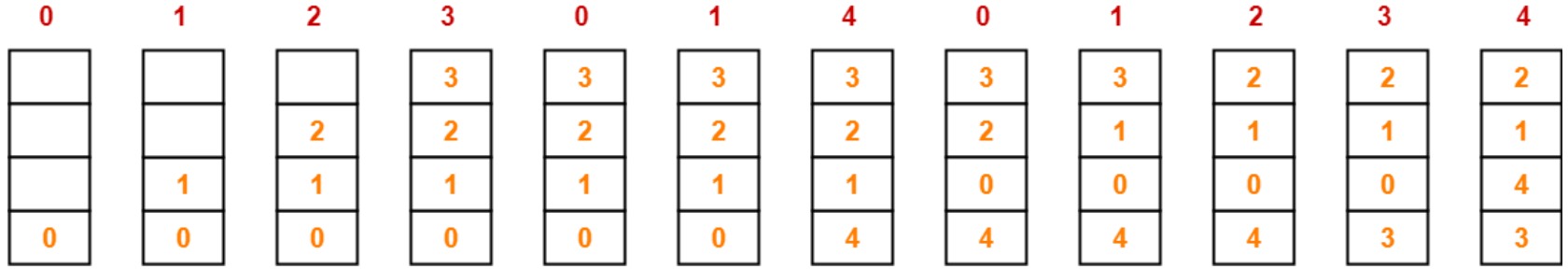
0 1 2 3 0 1 4 0 1 2 3 4

Belady's anomaly

- FIFO lacks the stack property, so more frames can mean more faults



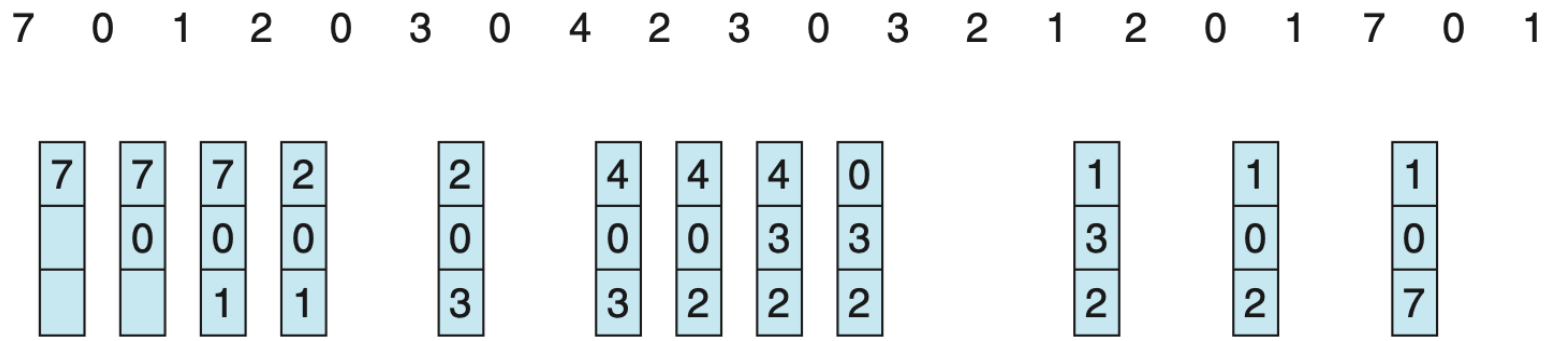
- Number of page faults with 3 frames = 9



- Number of page faults with 4 frames = 10

Page replacement algorithms: LRU

- We can’t see the future. What can we do?
  - → Look at the past
- Least Recently Used (LRU): evict the page unused for the longest time
  - → OPT looking backwards
  - → The evicted page might be needed right away



LRU (cont'd)

- LRU does not exhibit Belady's anomaly
- How can we implement LRU?
  - → Timestamp per entry: space, plus a search
  - → Ordered list: updates on every access
  - → Both impractical without special hardware
- Approximate it with a **reference bit**
  - → Set when the page is referenced
  - → Cleared periodically

Page replacement algorithms: others

- Second Chance: FIFO queue + reference bit
  - → Bit 0: replace the page
  - → Bit 1: clear it, move page to the back
  - → Downside?
    - Constantly moving pages in the list
- Clock: same idea, circular list
  - → Hand points at the oldest page
  - → Bit 0: replace; bit 1: clear and advance
  - → A good approximation of LRU

Page replacement algorithms: others (cont'd)

- Least Frequently Used (LFU): evict lowest use count
  - → Downside?
    - Heavy early use keeps a page forever
- Most Frequently Used (MFU): evict highest count
  - → Downside?
    - Evicts the pages you use most
- Both are poor: expensive and don't approximate OPT

Page replacement algorithms: summary

- OPT is optimal but impossible; LRU approximates it well
- Many algorithms, with different performance and hardware needs
- e.g., Linux: active/inactive lists (2Q-like)
  - → Driven by the hardware referenced bit
- Page replacement shows up at every level of the memory hierarchy
  - → We focus on disk/RAM: the biggest gap in access time

What isn’t clear?

Comments? Thoughts?

Page tables

Questions to consider

- Why is page table size a critical concern for memory efficiency?
- How can hardware support (like the TLB) improve address translation performance?
- How does context switching interact with the TLB, and what mechanisms address the correctness and performance problems it creates?

Page table access

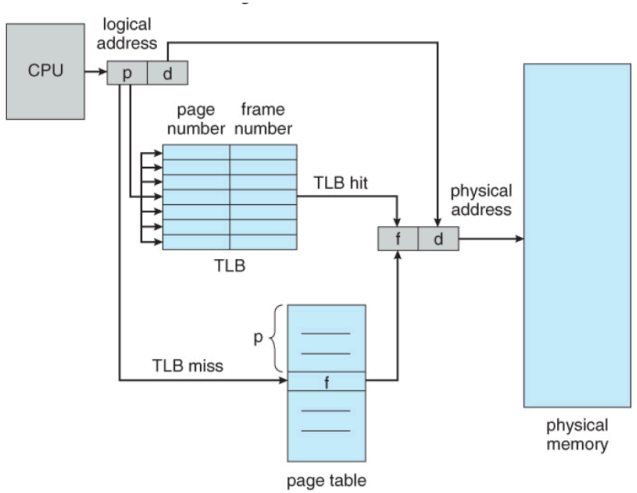
- Where is the page table stored?
  - → Old: dedicated registers (doesn't scale)
  - → Now: main memory, pointer in the PCB
- Downsides?
  - → Large tables use a lot of memory
  - → Two memory accesses per access
  - → How do we speed this up?
    - Hardware

Translation lookaside buffer (TLB)

- Uses **associative memory**
  - → What's special about associative (content-addressable) memory?
    - *Parallel search*: all keys checked at once
- Key (tag)–value pairs: a cache for the page table
- **Expensive**, so only a small number of entries

TLB (cont'd)

- CPU issues a logical address (e.g., load from address 100)
- On a **miss**, fetch the entry from the page table
  - → *Soft miss*: page is in memory
  - → *Hard miss*: page is on disk (page fault)



TLB (cont'd)

- On a **hit**, we get the physical address directly
  - → Hit: 1 memory access; miss: 2
- Hit ratio sets the **effective access time** (EAT)
- Example: 80% hits, 10ns per memory access
  - → EAT = $(.8 \times 10) + (.2 \times 20) = 12ns$
  - → 99% hits: $(.99 \times 10) + (.01 \times 20) = 10.1ns$
- Entries that must always stay (e.g., kernel)?
  - → "Wired down": never replaced

How good does the TLB need to be?

- Memory access: 10ns. A perfect TLB gives EAT = 10ns
- What hit rate keeps EAT within 5% of perfect (≤ 10.5ns)?
    1. Single-level page table (a miss costs 2 accesses)
    2. 4-level page table (a miss costs 5 accesses)

TLB (cont'd)

- Multiple processes, one small TLB. What happens on a context switch?
  - → Process 1's virtual page 1 ≠ process 2's virtual page 1
- Could **flush** the TLB on every switch
  - → Downside?
    - Lose the cache; rebuild it every time
- Other idea?
  - → Address-space identifiers (ASIDs): tag each TLB entry with its process

What isn’t clear?

Comments? Thoughts?

Exam 2 boundary

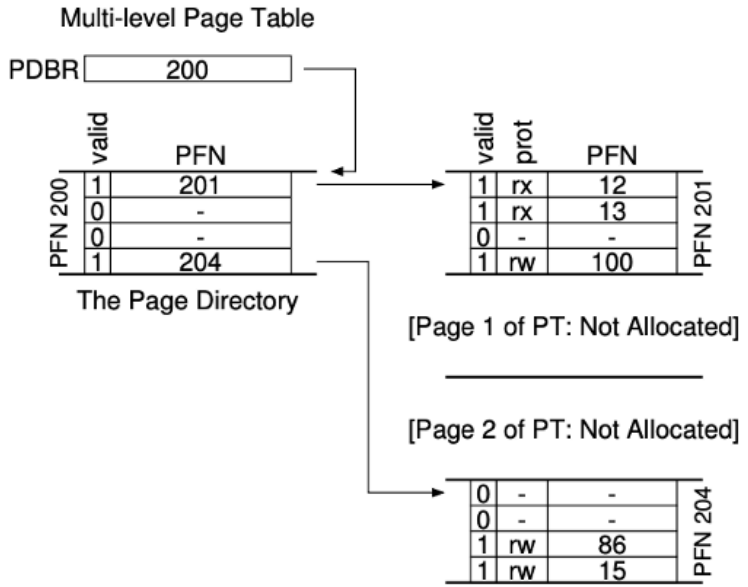
Page table structures

Questions to consider

- How can we organize page tables to handle large address spaces efficiently?
- What are the trade-offs between space overhead and lookup complexity?
- When and why would we choose alternative page table designs?

Page table structures

- Linear page tables are wasteful, especially with many unused pages (likely)
- Answer?
  - → Multilevel page tables

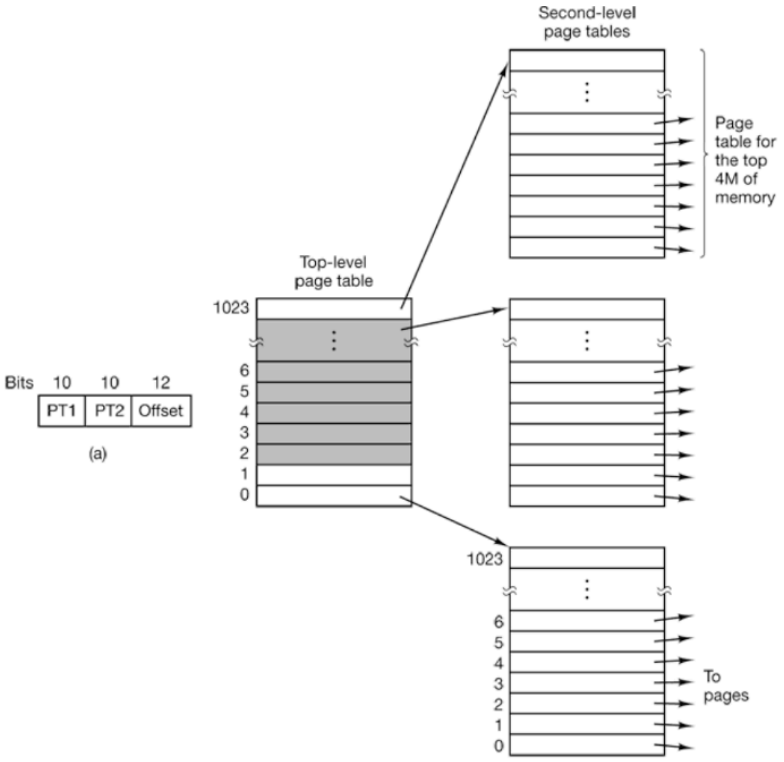


The rule that drives everything

- **Each page table fits in one page** (on x86 and most designs)
  - → Page directory and every second-level table: one 4KB page each
  - → One directory per process, plus a table per valid directory entry
  - → Why? The paging system can manage page tables like any other page
- This constraint sets everything else:
  - → entry size → entries per table → bits per level

Multilevel page tables

- Example: two-level page table
  - → 10-bit PT1, 10-bit PT2, 12-bit offset
- Top level points to 1024 second-level tables
- Each second-level table maps 1024 4KB pages



- Benefit?
  - → Unused second-level tables needn't exist
  - → Here: 4 tables (16KB), not 4MB

Two-level page table: address breakdown

Field	Bits	Meaning
PT1 (dir index)	10	which page table
PT2 (page index)	10	which entry in that table
Offset	12	byte within page

- **4-byte** entries → $4096/4 = 1024 = 2^{10}$ entries → **10-bit index**
- 2^{10} × 4B = 4KB = **exactly one page**
- Invalid directory entry: its page table **needn't exist**

Flat vs. two-level

- 32-bit addresses, 4KB pages, 4-byte entries
- A process uses:
  - → 4MB of code at the bottom of its address space
  - → 1 page of stack at the very top
- How much memory do its page tables take with a flat table? With the two-level table above?

Multilevel page tables (cont'd)

- Extends to more levels
- 64-bit doesn't use all 64 bits

  → x86-64: 48 bits, 256TB

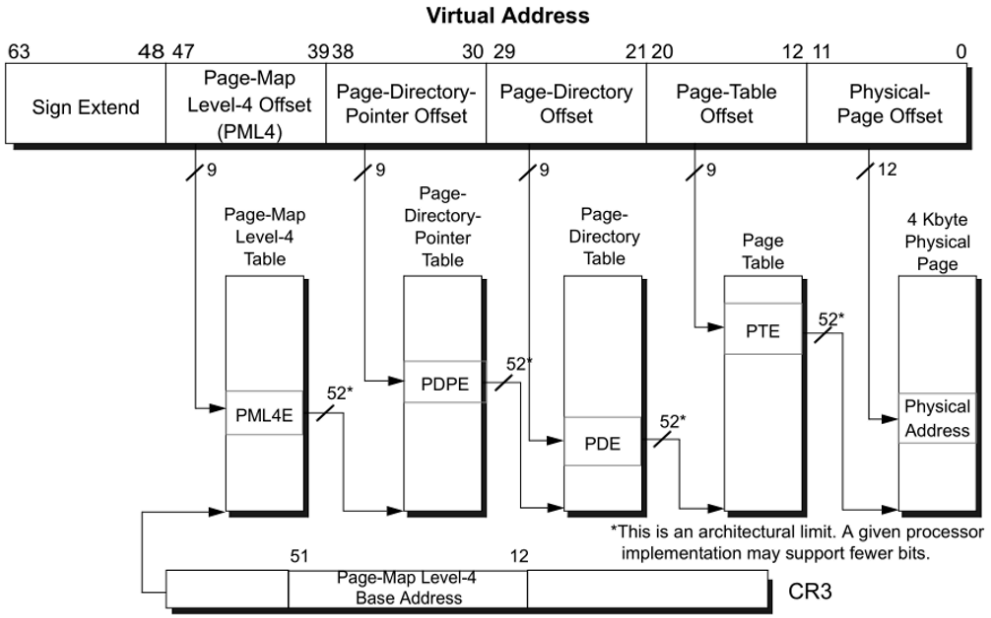
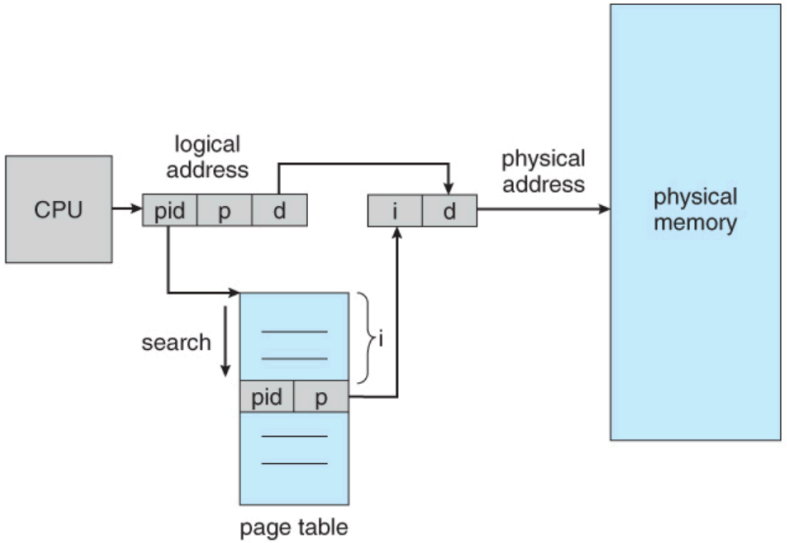


Figure 5-17. 4-Kbyte Page Translation—Long Mode

Inverted page tables

- So far: one page table per process
- Alternative: one system-wide table, one entry per physical frame (**inverted page table**)
  - → Entry: owning PID + virtual page number



Inverted page tables (cont'd)

- Smaller, but lookups are harder
  - → Naive lookup searches the whole table (slow)
- Fix: hash table index, plus a TLB for frequent lookups
- Rare on mainstream x86 and ARM
  - → Used by PowerPC and IA-64 (hashed tables)

What isn’t clear?

Comments? Thoughts?